

Lab 1:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.first_name, c.last_name, r.rental_date
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE c.customer_id = 1;
```

1. Roles

- **Outer (top):** **customer** table (Index Scan on primary key).
- **Inner (bottom):** **rental** table (Index Scan on **customer_id** foreign key).

2. Why Nested Loop?

- **Rows returned:** Exactly **1 row** from the **customer** table.
- **Why Hash Join is too expensive:** Building and sizing a hash table in memory for just 1 row introduces unnecessary startup cost and memory overhead. A direct B-tree index lookup on **rental** via Nested Loop is much faster and cheaper.

3. Efficiency (**loops = 1**)

- The inner scan executes once for each row emitted by the outer relation.
- Since the outer relation (**customer**) yields only **1 row**, the inner node is called only **1 time** (**loops = 1**).

lab 2:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.title, l.name AS language
FROM film f
JOIN language l ON f.language_id = l.language_id;
```

1. Identify Phases

- **Hashed (Built):** **language** table. The planner picks the smaller relation (typically 6 rows) to build the in-memory hash table.
- **Probed:** **film** table. The larger relation (1,000 rows) is sequentially scanned to probe against the hash table keys.

2. Memory Check

- **Memory Usage:** Typically **~9 kB** (with **Buckets: 1024**, **Batches: 1**). Because the **language** table contains only a handful of rows, the hash table footprint is minimal.

3. Hypothesis (SET work_mem = '64kB';)

- **Outcome:** The execution plan **remains unchanged** as a 1-batch in-memory Hash Join.
- **Reason:** The minimum **work_mem** in PostgreSQL is 64 kB, which is still much larger than the required ~9 kB memory footprint ($9\text{ kB} < 64\text{ kB}$). It does not spill to disk or increase batches because the hash table still fits entirely in memory.

lab 3

```
SET enable_hashjoin = OFF;  
SET enable_nestedloop = OFF;
```

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT f.film_id, f.title, i.inventory_id  
FROM film f  
JOIN inventory i ON f.film_id = i.film_id  
ORDER BY f.film_id;
```

1. Structure

- **No Sort node is used.**
- The planner performs an **Index Scan** on both tables (**film_pkey** on **film** and **idx_fk_film_id** on **inventory**). Because B-tree indexes store keys in sorted order, PostgreSQL reads the rows pre-sorted "for free" without needing an explicit sorting step.

```
RESET ALL;
```

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT f.film_id, f.title, i.inventory_id  
FROM film f  
JOIN inventory i ON f.film_id = i.film_id;
```

2. Compare (Reset & Remove ORDER BY)

- **Switches to Hash Join:** Yes, the planner reverts to a **Hash Join**.
- **Why:**
 - Without an **ORDER BY** clause, preserving sorted order is no longer required.
 - Reading entire tables using fast sequential scans (**Seq Scan**) and building an in-memory hash table incurs lower I/O overhead and total execution cost than traversing multiple B-tree index pages row-by-row.

